ĐẠI HỌC QUỐC GIA TP HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA ĐIỆN – ĐIỆN TỬ

BÁO CÁO BÀI TẬP LỚN 2
THIẾT KẾ VÀ KIỂM ĐỊNH HỆ THỐNG SỐ (EE3213)

Lớp: L01 – Học kỳ: 251 – Nhóm 06

Giảng viên hướng dẫn: Th.S Nguyễn Trung Hiếu
Sinh viên thực hiện:

Họ và tên	MSSV	Hoàn thành(%)
Lê Quang Tùng	2313812	100%
Vũ Thái Tuấn	2313773	100%
Vũ Tiến Tuấn	2313774	100%

Thành phố Hồ Chí Minh, tháng 1 năm 2026

MỤC LỤC

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI 1

1.1. Giới thiệu 1

1.2. Mục TIÊU của ĐỀ TÀI 1

CHƯƠNG 2: THIẾT KẾ 2

2.1. Phân tích yêu cầu 2

2.2. Lý thuyết 2

2.3. Cấu trúc module FFT 8 - points 4

2.3.1. Kiến trúc phân cấp tổng thể 4

2.3.2. Kiến trúc các module con 4

2.4. Code verilog 14

2.5. Kiểm định bằng assertion 34

2.5.1. CODE 34

2.5.2. Kết quả 43

2.6. Link video báo cáo thuyết trình 43

CHƯƠNG 1: GIỚI THIỆU ĐỀ TÀI

1.1. GIỚI THIỆU

Trong các hệ thống xử lý tín hiệu số hiện đại như thông tin vô tuyến, xử lý âm thanh, hình ảnh và radar, biến đổi Fourier nhanh (Fast Fourier Transform – FFT) đóng vai trò quan trọng trong việc chuyển đổi tín hiệu từ miền thời gian sang miền tần số. So với phép biến đổi Fourier rời rạc (DFT) thông thường, thuật toán FFT giúp giảm đáng kể độ phức tạp tính toán, từ đó nâng cao hiệu năng và tốc độ xử lý của hệ thống.

Xuất phát từ yêu cầu đó, đề tài tập trung nghiên cứu và thiết kế bộ FFT 8-point trên nền tảng phần cứng số. FFT 8-point là cấu hình cơ bản, có cấu trúc rõ ràng, phù hợp để tìm hiểu nguyên lý hoạt động của thuật toán FFT cũng như cách ánh xạ thuật toán này sang kiến trúc phần cứng gồm các khối như ROM lưu dữ liệu, khối chuyển đổi nối tiếp – song song (SIPO), các khối butterfly, khối trễ (delay) và hệ thống pipeline đồng bộ theo xung clock.

1.2. MỤC TIÊU CỦA ĐỀ TÀI

Mục tiêu của đề tài là hiểu và triển khai được thuật toán FFT 8-point dưới dạng phần cứng số, thông qua việc phân tích nguyên lý FFT và thiết kế các khối chức năng tương ứng. Cụ thể, đề tài hướng đến việc xây dựng các khối cơ bản như ROM lưu 8 mẫu dữ liệu đầu vào, khối SIPO để chuyển dữ liệu nối tiếp sang song song, các khối butterfly thực hiện phép cộng và trừ theo từng stage của FFT, cũng như các khối delay và thanh ghi D flip-flop nhằm đảm bảo đồng bộ dữ liệu và phù hợp với kiến trúc pipeline.

Thông qua quá trình thiết kế, mô phỏng và kiểm tra, đề tài giúp làm rõ mối liên hệ giữa thuật toán FFT về mặt toán học và cách hiện thực hóa thuật toán đó trên phần cứng, đồng thời tạo nền tảng cho việc mở rộng sang các cấu hình FFT có số điểm lớn hơn trong các ứng dụng thực tế.

CHƯƠNG 2: THIẾT KẾ

2.1. Phân tích yêu cầu

Yêu cầu tối thiểu: input khi tính FFT là số nguyên 32 bit Q(32.0), khi nhân với số phức (fixed point multiplier) xấp xỉ làm tròn 0.7071 thành fixed point number 32 bit: 0.7071 = 32'hB504F333 Q(0.32) với kết quả sau phép nhân là số fixed point 32 bit Q(16.16).

2.2. Lý thuyết

Phép biến đổi Fourier rời rạc (Discrete Fourier Transform – DFT) cho phép chuyển một dãy tín hiệu rời rạc trong miền thời gian sang miền tần số, tuy nhiên việc tính trực tiếp DFT có độ phức tạp O(N^2), không hiệu quả khi số điểm N lớn. Thuật toán Fast Fourier Transform (FFT) được phát triển nhằm giảm độ phức tạp tính toán xuống còn O(Nlog2N) bằng cách khai thác tính đối xứng và tuần hoàn của các hệ số mũ phức.

Đối với FFT 8-point (N=8), thuật toán radix-2 chia bài toán DFT 8 điểm thành các bài toán con nhỏ hơn thông qua phương pháp chia để trị. Cụ thể, đa thức biểu diễn tín hiệu đầu vào được tách thành hai phần chẵn và lẻ, từ đó biến đổi DFT 8-point thành các phép biến đổi 4-point, 2-point và cuối cùng là các phép toán cộng, trừ và nhân với các hệ số xoay phức (twiddle factors). Quá trình này được thực hiện qua ba stage butterfly liên tiếp, trong đó mỗi butterfly thực hiện phép cộng – trừ giữa hai mẫu dữ liệu và, ở các stage sau, nhân với các twiddle factor tương ứng.

Nhờ cấu trúc butterfly và việc tái sử dụng kết quả trung gian, FFT 8-point giảm đáng kể số phép nhân và phép cộng so với DFT trực tiếp, đồng thời rất phù hợp cho việc hiện thực trên phần cứng số như FPGA hoặc ASIC do có kiến trúc song song, đều đặn và dễ pipeline.

Giai đoạn 1 — Tách chẵn – lẻ (even / odd)

$$a_0 = x_0 + x_4$$

$$a_4 = x_0 - x_4$$

$$a_1 = x_1 + x_5$$

$$a_5 = x_1 - x_5$$

$$a_2 = x_2 + x_6$$

$$a_6 = x_2 - x_6$$

$$a_3 = x_3 + x_7$$

$$a_7 = x_3 - x_7$$

Giai đoạn 2 — Nhân twiddle

Nhánh trên (W = 1)

$$b_0 = a_0 + a_2$$

$$b_2 = a_0 - a_2$$

$$b_1 = a_1 + a_3$$

$$b_3 = a_1 - a_3$$

Nhánh dưới (W = -j)

$$b_4 = a_4 + a_6$$

$$b_6 = (a_4 - a_6).(-j)$$

$$b_5 = a_5 + a_7$$

$$b_7 = (a_5 - a_7).(-j)$$

Giai đoạn 3 — Nhân twiddle đầy đủ

Twiddle factor

$$W_8^0 = 1$$

$$W_8^1 = \frac{1}{\sqrt{2}}(1-j)$$

$$W_8^2 = -j$$

$$W_8^3 = \frac{1}{\sqrt{2}}(-1-j)$$

$$y_0 = b_0 + b_1$$

$$y_4 = b_0 - b_1$$

$$y_2 = b_2 + b_3$$

$$y_6 = (b_2 - b_3)(-j)$$

$$y_1 = b_4 + b_5 . W_8^1$$

$$y_5 = b_4 - b_5 . W_8^1$$

$$y_3 = b_6 + b_7.W_8^3$$
$$y_7 = b_6 - b_7.W_8^3$$

2.3. Cấu trúc module FFT 8 - points

2.3.1. Kiến trúc phân cấp tổng thể

Kiến trúc bộ FFT 8-point được xây dựng theo mô hình phân tầng (hierarchical architecture), trong đó mỗi tầng đảm nhiệm một chức năng rõ ràng và độc lập, giúp quá trình thiết kế, kiểm tra và mở rộng hệ thống trở nên thuận lợi. Ở tầng đầu vào, khối ROM được sử dụng để lưu trữ các mẫu tín hiệu đầu vào hoặc các hệ số thử nghiệm, đóng vai trò nguồn dữ liệu cho toàn bộ hệ thống. Dữ liệu từ ROM được xuất ra tuần tự theo xung clock và được đưa vào khối chuyển đổi nối tiếp–song song (Serial-to-Parallel, SIPO). Khối SIPO có nhiệm vụ gom đủ 8 mẫu liên tiếp và phân bố chúng thành các ngõ ra song song x0x_0x0 đến x7x_7x7, tạo điều kiện cho các phép toán butterfly được thực hiện đồng thời.

Sau tầng tiền xử lý dữ liệu, khối xử lý chính của FFT bao gồm ba tầng butterfly tương ứng với ba stage của thuật toán radix-2 DIT. Ở tầng butterfly thứ nhất (BF1), các cặp dữ liệu cách nhau N/2N/2N/2 được ghép lại để thực hiện các phép cộng và trừ, nhằm tách tín hiệu thành các thành phần chẵn và lẻ mà không cần sử dụng phép nhân phức. Tầng butterfly thứ hai (BF2) tiếp tục xử lý các kết quả trung gian bằng các phép cộng, trừ và bắt đầu nhân với các hệ số xoay đơn giản, điển hình là −j-j−j, từ đó tạo ra các thành phần phức cần thiết cho quá trình biến đổi. Đến tầng butterfly thứ ba (BF3), các kết quả được ghép lần cuối và nhân với các twiddle factor đầy đủ của FFT 8-point, hoàn thiện việc chuyển đổi tín hiệu sang miền tần số.

Cách tổ chức kiến trúc theo từng tầng chức năng như vậy giúp giảm độ phức tạp của từng module con, tối ưu hóa tài nguyên phần cứng và cho phép thiết kế hoạt động hiệu quả theo kiểu song song và pipeline. Đồng thời, kiến trúc phân tầng này phản ánh trực tiếp cấu trúc toán học của FFT, tạo sự tương ứng rõ ràng giữa thuật toán và phần cứng, từ đó nâng cao tính dễ hiểu, khả năng kiểm chứng và khả năng tái sử dụng của hệ thống FFT 8-point.

2.3.2. Kiến trúc các module con

2.3.2.1. Module ROM

a) Input và Output (mô tả)

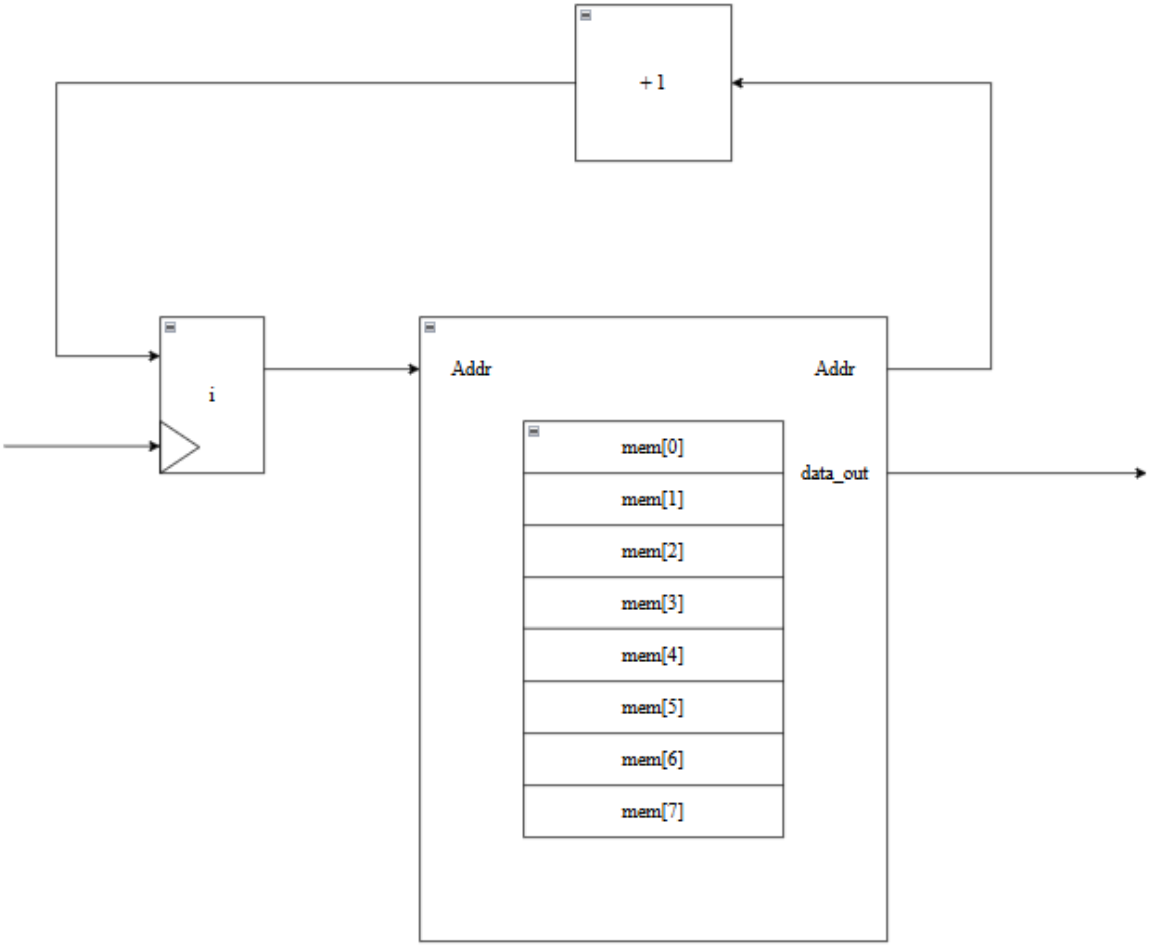
Module ROM có nhiệm vụ lưu trữ các mẫu tín hiệu đầu vào hoặc các hệ số kiểm thử phục vụ cho quá trình đánh giá và mô phỏng bộ FFT 8-point. Dữ liệu trong ROM

được khởi tạo sẵn và xuất ra theo thứ tự tuần tự dưới sự điều khiển của xung clock nội bộ, không yêu cầu địa chỉ truy cập từ bên ngoài. Cách tổ chức này giúp đơn giản hóa giao tiếp và đảm bảo dữ liệu được cung cấp đúng thứ tự thời gian cho khối xử lý phía sau. Trong hệ thống FFT, ROM đóng vai trò là nguồn phát dữ liệu nối tiếp, phù hợp với các kiến trúc xử lý theo dòng (streaming).

Tên cổng	Hướng	Độ rộng	Chức năng
clk	Input	1 bit	Xung clk
rst_n	Input	1 bit	Reset tích cực thấp
Data_out	Output	32 bit	Giá trị ghi ra từ trong thanh ghi ROM

Bảng 2.1. Cổng I/O của module ROM

b) Block diagram



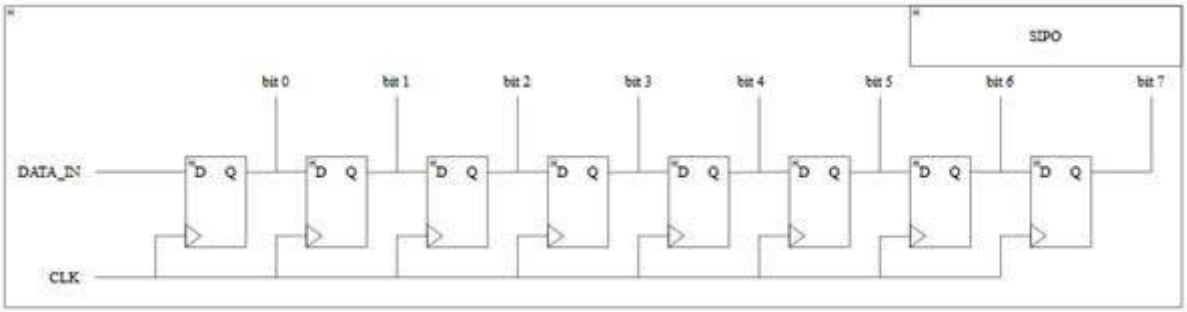
2.3.2.2. Module SIPO (serial in parallel out)

Module SIPO thực hiện chức năng chuyển đổi dữ liệu từ dạng nối tiếp sang dạng song song. Cụ thể, SIPO tiếp nhận từng mẫu dữ liệu 32-bit từ ROM tại mỗi chu kỳ clock và lưu trữ chúng trong chuỗi các thanh ghi dịch. Sau khi nhận đủ 8 mẫu liên tiếp, module xuất ra đồng thời 8 ngõ ra song song x0đến x7, tương ứng với 8 điểm đầu vào của FFT. Khối SIPO đảm bảo dữ liệu được căn chỉnh đúng thời điểm, tạo điều kiện cho các phép toán butterfly được thực hiện song song ở các stage tiếp theo.

Tên cổng	Hướng	Độ rộng	Chức năng
clk	Input	1 bit	Xung clk
rst_n	Input	1 bit	Reset tích cực thấp
Data_out	Output	32 bit	Giá trị ghi ra từ trong thanh ghi ROM

Bảng 2.2. Cổng I/O của module srl

a) Block diagram



2.3.2.3. Module butterfly stage 1 (BF1)

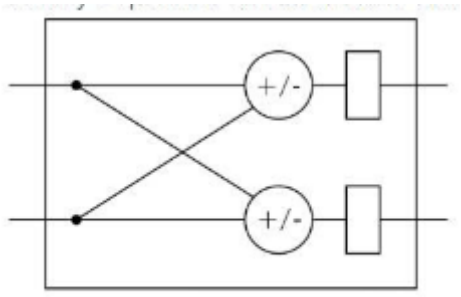
a) Input và Output

Butterfly stage 1 thực hiện bước xử lý đầu tiên của thuật toán FFT 8-point radix-2. Module này ghép các cặp mẫu dữ liệu cách nhau N/2 và thực hiện các phép cộng và trừ tương ứng nhằm tách tín hiệu thành các thành phần chẵn và lẻ. Do hệ số xoay ở stage này bằng 1, BF1 không cần sử dụng phép nhân phức, giúp giảm đáng kể tài nguyên phần cứng. Kết quả của BF1 là các giá trị trung gian đã được tái cấu trúc, làm đầu vào cho stage tiếp theo.

Tên cổng	Hướng	Độ rộng	Chức năng
a	Input	24 bit	Mantissa của toán hạng lớn
b	Input	24 bit	Mantissa của toán hạng nhỏ đã căn chỉnh
cin	Input	1 bit	Bit nhớ vào (luôn bằng 0 trong thiết kế)
signa	Input	1 bit	Dấu hiệu dụng của toán hạng lớn
signb	Input	1 bit	Dấu hiệu dụng của toán hạng nhỏ
out	Output	26 bit	Kết quả cộng/trừ mantissa

Bảng 2.3. Cổng I/O của module flp_adder

b) Block diagram



Butter Fly (BF1)

2.3.2.4. Module butter fly stage 2 (BF2)

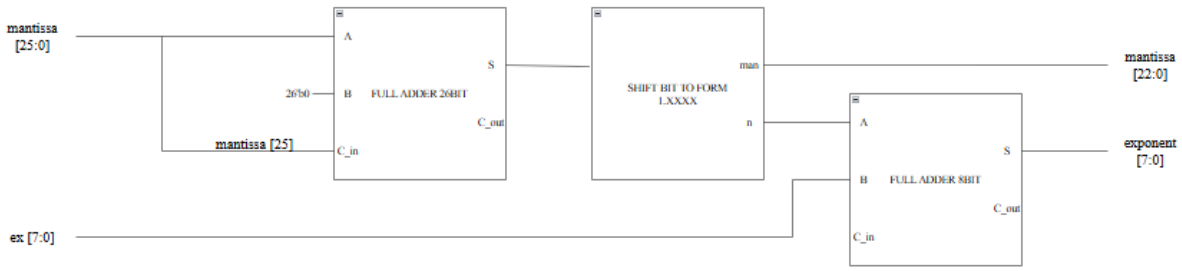
a) Input và Output

Butterfly stage 2 tiếp tục xử lý các dữ liệu trung gian từ BF1 bằng cách thực hiện các phép cộng, trừ và nhân với các twiddle factor đơn giản, điển hình là −j. Module này bắt đầu tạo ra các thành phần phức của tín hiệu, đóng vai trò cầu nối giữa các phép toán thuần số thực ở stage đầu và các phép toán phức ở stage cuối. Việc sử dụng hệ số xoay đơn giản giúp BF2 đạt được sự cân bằng giữa độ chính xác và mức tiêu thụ tài nguyên phần cứng.

Tên cổng	Hướng	Độ rộng	Chức năng
ex	Input	8 bit	Exponent của toán hạng lớn
mantissa	Input	26 bit	Kết quả cộng/trừ mantissa
rs_sign	Output	1 bit	Bit dấu sau chuẩn hóa
rs_exponent	Output	8 bit	Exponent sau chuẩn hóa
rs_mantissa	Output	23 bit	Mantissa sau chuẩn hóa

Bảng 2.4. Cổng I/O của module normalization

b) Block diagram

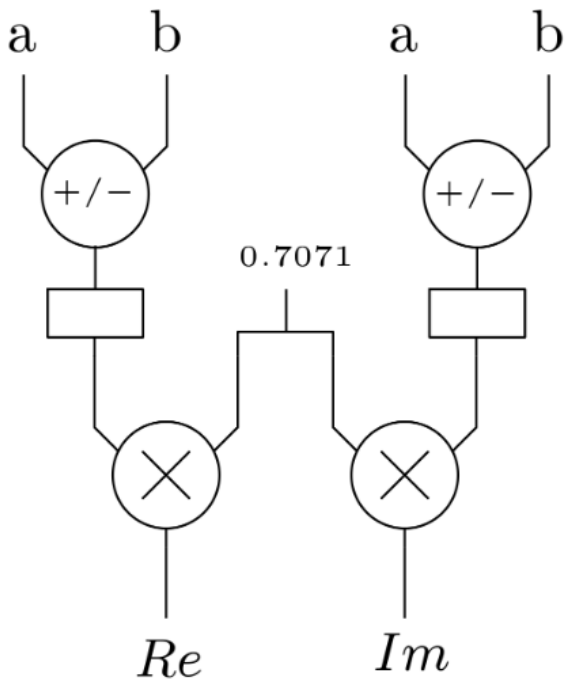


2.3.2.5. Module Complex Multiplier (CM)

a) Input và Output

Tên cổng	Hướng	Độ rộng	Chức năng
a	Input	32	Ngõ vào dữ liệu (toán hạng 1)
b	Input	32	Ngõ vào dữ liệu (toán hạng 2)
re	Output	32	Ngõ ra phần thực (lấy từ tích với hằng W = 0.7071: 32'hB504F333 (Q0.32))
im	Output	32	Ngõ ra phần ảo (trong code đang gán tương tự nhánh nhân W)

b) Block diagram

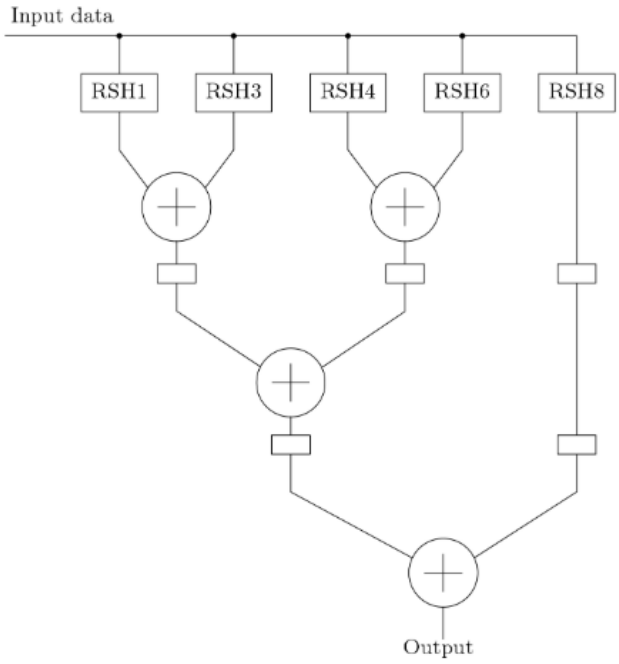


2.3.2.6. Module Constant Multiplier

a) Input và Output

Tên cổng	Hướng	Độ rộng	Chức năng
a	Input	32	Toán hạng A (fixed-point)
b	Input	32	Toán hạng B. Khi nối b = W thì block này đóng vai trò **constant multiplier**
out	Output	64	Kết quả nhân 64-bit (trước khi cắt/scale)

b) Blok diagram



2.3.2.7. Control Path of The FFT Processor

Tên cổng	Hướng	Độ rộng	Chức năng
clk	Input	1	Clock hệ thống
rst_n	Input	1	Reset active-low
start	Input	1	Xung bắt đầu 1 frame FFT
en1	Output	1	Enable ROM + S2P bắt đầu nhận 8 mẫu
tc1	Output	1	Terminal count sau khi đủ 8 clock (xong S2P)
en2	Output	1	Enable "đếm latency FFT" / pha tiếp theo
tc2	Output	1	Đồng bộ với output FFT đầu tiên; dùng làm load cho P2S
en3	Output	1	Dùng làm p2s cho P2S để bắt đầu xuất nối tiếp
tc3 *(tuỳ chọn)*	Output	1	Terminal count kết thúc serialize (xong xuất 8 mẫu)
rom_addr *(tuỳ chọn)*	Output	3	Địa chỉ ROM 0..7 để đọc 8 mẫu (nếu bạn có ROM)
done *(tuỳ chọn)*	Output	1	Báo hoàn tất 1 frame FFT

kết quả các số y là số thực (ngoại trừ y3 và y7 là kết quả cho số ảo)

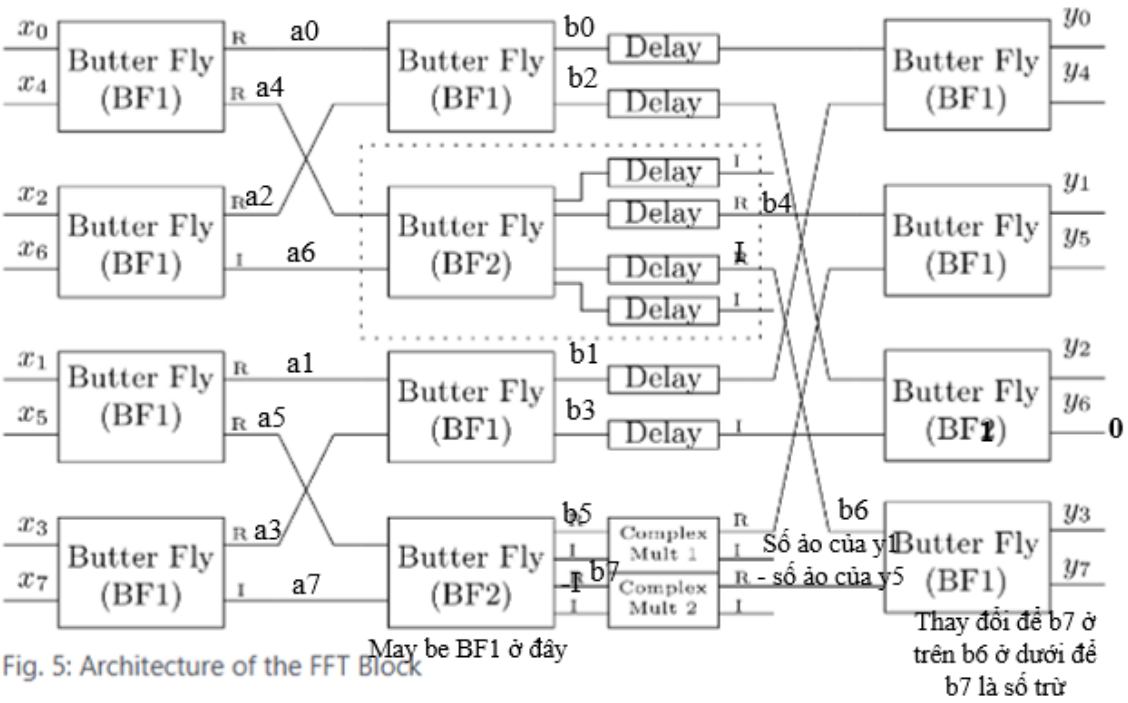
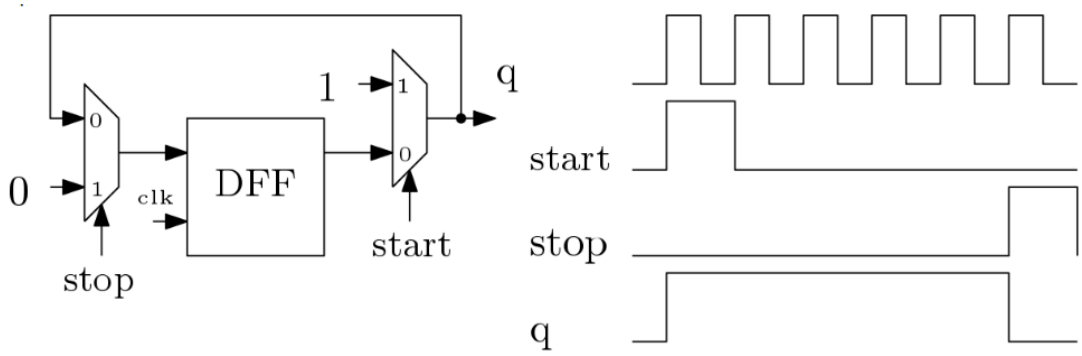


Fig. 5: Architecture of the FFT Block

a) Phase Generation (PG) Block:

Khối pg tạo một tín hiệu pha/cho phép q để "mở" một giai đoạn hoạt động trong hệ thống. khi nhận xung start, q được set lên 1 và giữ ở mức 1 trong suốt giai đoạn đó. khi nhận xung stop, q được reset về 0 để kết thúc giai đoạn. nói cách khác, pg biến các xung điều khiển rời rạc (start/stop) thành một cửa sổ enable ổn định theo thời gian cho các khối phía sau.

- Block diagram



b) Loadable Counter

Loadable counter là bộ đếm có giới hạn (limit/lmt) dùng để đếm số chu kỳ clock trong một giai đoạn khi enable đang bật. khi enable = 1, bộ đếm tăng dần và khi đạt tới giá trị giới hạn, nó phát tín hiệu tc (terminal count) để báo "đã đủ số chu kỳ cần thiết" và thường dùng tc để chuyển sang pha tiếp theo hoặc tạo xung điều khiển như load/p2s. trong kiến trúc fft, counter thường dùng để đếm 8 mẫu vào/ra hoặc canh độ trễ của khối fft.

2.4. CODE VERILOG

```
// --------------- full adder ---------------
module full_adder(
  input logic a,
  input logic b,
  input logic c_in,
  output logic sum,
  output logic c_out
);
  assign sum   = a ^ b ^ c_in;
  assign c_out = a  &  b | (a ^ b) &   c_in;
endmodule


// --------------- full adder 32 bit ---------------
module full_adder_32bit (
  input  logic [31:0] a,
  input  logic [31:0] b,
  input  logic        c_in,
  output logic [31:0] sum,
```

```
        output logic        c_out
      );

        logic [30:0] carry;   // các carry trung gian

        full_adder                                                    fa0
(.a(a[0]),  .b(b[0]),  .c_in(c_in),      .sum(sum[0]),  .c_out(carry[0]));
        full_adder                                                    fa1
(.a(a[1]),  .b(b[1]),  .c_in(carry[0]), .sum(sum[1]),  .c_out(carry[1]));
        full_adder                                                    fa2
(.a(a[2]),  .b(b[2]),  .c_in(carry[1]), .sum(sum[2]),  .c_out(carry[2]));
        full_adder                                                    fa3
(.a(a[3]),  .b(b[3]),  .c_in(carry[2]), .sum(sum[3]),  .c_out(carry[3]));
        full_adder                                                    fa4
(.a(a[4]),  .b(b[4]),  .c_in(carry[3]), .sum(sum[4]),  .c_out(carry[4]));
        full_adder                                                    fa5
(.a(a[5]),  .b(b[5]),  .c_in(carry[4]), .sum(sum[5]),  .c_out(carry[5]));
        full_adder                                                    fa6
(.a(a[6]),  .b(b[6]),  .c_in(carry[5]), .sum(sum[6]),  .c_out(carry[6]));
        full_adder                                                    fa7
(.a(a[7]),  .b(b[7]),  .c_in(carry[6]), .sum(sum[7]),  .c_out(carry[7]));
        full_adder                                                    fa8
(.a(a[8]),  .b(b[8]),  .c_in(carry[7]), .sum(sum[8]),  .c_out(carry[8]));
        full_adder                                                    fa9
(.a(a[9]),  .b(b[9]),  .c_in(carry[8]), .sum(sum[9]),  .c_out(carry[9]));
        full_adder                                                   fa10
(.a(a[10]), .b(b[10]), .c_in(carry[9]), .sum(sum[10]), .c_out(carry[10]));
```

```
full_adder fa11
(.a(a[11]), .b(b[11]), .c_in(carry[10]),.sum(sum[11]), .c_out(carry[11]));
full_adder fa12
(.a(a[12]), .b(b[12]), .c_in(carry[11]),.sum(sum[12]), .c_out(carry[12]));
full_adder fa13
(.a(a[13]), .b(b[13]), .c_in(carry[12]),.sum(sum[13]), .c_out(carry[13]));
full_adder fa14
(.a(a[14]), .b(b[14]), .c_in(carry[13]),.sum(sum[14]), .c_out(carry[14]));
full_adder fa15
(.a(a[15]), .b(b[15]), .c_in(carry[14]),.sum(sum[15]), .c_out(carry[15]));
full_adder fa16
(.a(a[16]), .b(b[16]), .c_in(carry[15]),.sum(sum[16]), .c_out(carry[16]));
full_adder fa17
(.a(a[17]), .b(b[17]), .c_in(carry[16]),.sum(sum[17]), .c_out(carry[17]));
full_adder fa18
(.a(a[18]), .b(b[18]), .c_in(carry[17]),.sum(sum[18]), .c_out(carry[18]));
full_adder fa19
(.a(a[19]), .b(b[19]), .c_in(carry[18]),.sum(sum[19]), .c_out(carry[19]));
full_adder fa20
(.a(a[20]), .b(b[20]), .c_in(carry[19]),.sum(sum[20]), .c_out(carry[20]));
full_adder fa21
(.a(a[21]), .b(b[21]), .c_in(carry[20]),.sum(sum[21]), .c_out(carry[21]));
full_adder fa22
(.a(a[22]), .b(b[22]), .c_in(carry[21]),.sum(sum[22]), .c_out(carry[22]));
full_adder fa23
(.a(a[23]), .b(b[23]), .c_in(carry[22]),.sum(sum[23]), .c_out(carry[23]));
full_adder fa24
(.a(a[24]), .b(b[24]), .c_in(carry[23]),.sum(sum[24]), .c_out(carry[24]));
```

```
        full_adder                                                    fa25
(.a(a[25]), .b(b[25]), .c_in(carry[24]),.sum(sum[25]), .c_out(carry[25]));
        full_adder                                                    fa26
(.a(a[26]), .b(b[26]), .c_in(carry[25]),.sum(sum[26]), .c_out(carry[26]));
        full_adder                                                    fa27
(.a(a[27]), .b(b[27]), .c_in(carry[26]),.sum(sum[27]), .c_out(carry[27]));
        full_adder                                                    fa28
(.a(a[28]), .b(b[28]), .c_in(carry[27]),.sum(sum[28]), .c_out(carry[28]));
        full_adder                                                    fa29
(.a(a[29]), .b(b[29]), .c_in(carry[28]),.sum(sum[29]), .c_out(carry[29]));
        full_adder                                                    fa30
(.a(a[30]), .b(b[30]), .c_in(carry[29]),.sum(sum[30]), .c_out(carry[30]));
        full_adder                                                    fa31
(.a(a[31]), .b(b[31]), .c_in(carry[30]),.sum(sum[31]), .c_out(c_out));


      endmodule


      // --------------- sub 32 bit ---------------
      module sub_32bit (
        input  logic [31:0] a,
        input  logic [31:0] b,
        output logic [31:0] sum,
        output logic        c_out
      );
        logic [31:0] b_1;
        assign b_1 = ~b;
```

```
  full_adder_32bit fa1(
    .a     (a),
    .b     (b_1),
    .c_in  (1'b1),
    .sum   (sum),
    .c_out (c_out)
  );
endmodule

// --------------- BF_1 ---------------
module butter_fly_1 (
  input logic [31:0] a,
  input logic [31:0] b,
  output logic [31:0] sum,
  output logic [31:0] sub
);
  full_adder_32bit fa2(
    .a      (a),
    .b      (b),
    .c_in   (1'b0),
    .sum    (sum),
    .c_out  ()
  );
```

```
  sub_32bit sub1(
    .a      (a),
    .b      (b),
    .sum    (sub),
    .c_out  ()
  );
endmodule


// --------------------- BF2 ------------------
module butter_fly_2 (
  input logic [31:0] r,
  input logic [31:0] i,
  output logic [31:0] r1,
  output logic [31:0] r2,
  output logic [31:0] i1,
  output logic [31:0] i2         // i2 = -i1;
);
  assign r1 = r;
  assign r2 = r;
  assign i1 = i;

  sub_32bit u1 (
    .a     (32'd0),
    .b     (i),
    .sum   (i2),
```

```
    .c_out ()
  );

endmodule

module mul_fix_point (
  input  logic [31:0] a,
  input  logic [31:0] b,
  output logic [63:0] out
);

  logic [63:0] acc;
  logic [63:0] partial [31:0];

  genvar i;
  generate
    for (i = 0; i < 32; i++) begin : GEN_PARTIAL
      assign partial[i] = b[i] ? ({{32{1'b0}}, a} << i) : 64'd0;
    end
  endgenerate

  // cộng dồn (combinational)
  integer k;
  always_comb begin
    acc = 64'd0;
```

```
    for (k = 0; k < 32; k++) begin
      acc = acc + partial[k];
    end
  end

  assign out = acc;

endmodule

// --------------------- complex multiplier 1 ------------------
module cm1 (
  input logic [31:0] a,
  input logic [31:0] b,
  output logic [31:0] re,
  output logic [31:0] im
);
  logic [31:0] tmp1;

  full_adder_32bit fa1(
    .a      (a),
    .b      (b),
    .c_in   (1'b0),
    .sum    (tmp1),
    .c_out  ()
  );
```

```
  logic [63:0] tmp2,tmp3;

  // 1/sqrt(2) = 0.7071 (Q0.32)
  localparam logic signed [31:0] W = 32'hB504F333;

  mul_fix_point mul1(
    .a     (tmp1),
    .b     (W),
    .out   (tmp2)
  );


  mul_fix_point mul2(
    .a     (tmp1),
    .b     (W),
    .out   (tmp3)
  );

  assign re = tmp2 [47:16];
  assign im = tmp3 [47:16];

endmodule


// --------------------- complex multiplier 2 ------------------
```

```
module cm2 (
  input logic [31:0] a,
  input logic [31:0] b,
  output logic [31:0] re,
  output logic [31:0] im
);
  logic [31:0] tmp1;

  sub_32bit sub1(
    .a      (a),
    .b      (b),
    .sum    (tmp1),
    .c_out  ()
  );
  logic [63:0] tmp2,tmp3;

  // 1/sqrt(2) = 0.7071 (Q0.32)
  localparam logic signed [31:0] W = 32'hB504F333;

  mul_fix_point mul3(
    .a     (tmp1),
    .b     (W),
    .out   (tmp2)
  );
```

```
  mul_fix_point mul4(
    .a     (tmp1),
    .b     (W),
    .out   (tmp3)
  );

  assign re = tmp2 [47:16];
  assign im = tmp3 [47:16];


endmodule

// ------------------ delay -----------------
module delay (
  input  logic            clk,
  input  logic            rst_n,    // reset active LOW
  input  logic signed [31:0] din,
  output logic signed [31:0] dout
);
// có 8 biến nên delay 9 clk để tính đúng kết quả
  logic signed [31:0] pipe [0:8];

  integer i;
  always_ff @(posedge clk or negedge rst_n) begin
```

```
    if (!rst_n) begin
      for (i = 0; i < 9; i++)
        pipe[i] <= '0;
    end else begin
      pipe[0] <= din;
      for (i = 1; i < 9; i++)
        pipe[i] <= pipe[i-1];
    end
  end

  assign dout = pipe[8];

endmodule



// ------------------ 8_point_fft -----------------
module fft (
  input logic clk,
  input logic rst_n,
  input logic [31:0] x0,x1,x2,x3,x4,x5,x6,x7,
  output logic [31:0] re0,re1,re2,re3,re4,re5,re6,re7,
  output logic [31:0] im0,im1,im2,im3,im4,im5,im6,im7
);
```

```
logic [31:0] a0,a4;
butter_fly_1 bf1_1 (
  .a(x0),
  .b(x4),
  .sum(a0),
  .sub(a4)
);

logic [31:0] a2,a6;
butter_fly_1 bf1_2 (
  .a(x2),
  .b(x6),
  .sum(a2),
  .sub(a6)
);

logic [31:0] a1,a5;
butter_fly_1 bf1_3 (
  .a(x1),
  .b(x5),
  .sum(a1),
  .sub(a5)
);

logic [31:0] a3,a7;
```

```
butter_fly_1 bf1_4 (
  .a(x3),
  .b(x7),
  .sum(a3),
  .sub(a7)
);

// tầng 2

logic [31:0] b0,b2;
butter_fly_1 bf1_5 (
  .a(a0),
  .b(a2),
  .sum(b0),
  .sub(b2)
);

logic [31:0] b4,b6;
butter_fly_1 bf1_10 (
  .a(a4),
  .b(a6),
  .sum(b4),
  .sub(b6)  // phần ảo chưa nhân -1
);
```

```
logic [31:0] b1,b3;
butter_fly_1 bf1_6 (
  .a(a1),
  .b(a3),
  .sum(b1),
  .sub(b3)
);

logic [31:0] b5,b7;
butter_fly_1 bf1_7 (
  .a(a5),
  .b(a7),
  .sum(b5),
  .sub(b7)  // phần ảo chua nhân -1
);

logic [31:0] re_b5,im_b5;
cm1 cm1(
  .a   (b5),
  .b   (32'd0),
  .re  (re_b5),
  .im  (im_b5)   // giữ nguyên: phần ảo của X1, nhân -1: phần ảo của x5
);

logic [31:0] re_b7,im_b7;
```

```
cm2 cm2(
  .a   (b7),
  .b   (32'd0),
  .re  (re_b7),  // giữ nguyên: phần thực của x7, nhân -1: phần thực của x3
  .im  (im_b7)
);



// tầng 3: delay

logic [31:0] tmp0,tmp2;
delay d1 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (b0),
  .dout  (tmp0)
);

delay d2 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (b2),
  .dout  (tmp2)
);
```

```
logic [31:0] tmp4,tmp6;
delay d3 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (b4),
  .dout  (tmp4)
);

delay d4 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (b6),
  .dout  (tmp6)
);


logic [31:0] tmp1,tmp3;
delay d5 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (b1),
  .dout  (tmp1)
);
```

```
delay d6 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (b3),
  .dout  (tmp3)
);


logic [31:0] tmp5,tmp7;
delay d7 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (re_b5),
  .dout  (tmp5)
);

delay d8 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (im_b7),
  .dout  (tmp7)
);

logic [31:0] tmp7_re,tmp5_im;
delay d9 (
```

```
  .clk   (clk),
  .rst_n (rst_n),
  .din   (re_b7),
  .dout  (tmp7_re)
);

delay d10 (
  .clk   (clk),
  .rst_n (rst_n),
  .din   (im_b5),
  .dout  (tmp5_im)
);

// tầng 4: kết quả
logic [31:0] rs0,rs4;
butter_fly_1 bf1_8 (
  .a(tmp0),
  .b(tmp1),
  .sum(rs0),
  .sub(rs4)
);

logic [31:0] rs1,rs5;
butter_fly_1 bf1_9 (
  .a(tmp4),
```

```
  .b(tmp5),
  .sum(rs1),
  .sub(rs5)
);

logic [31:0] rs2,rs6;
butter_fly_1 bf1_11 (
  .a(tmp2),
  .b(tmp3),
  .sum(rs2),
  .sub(rs6)  // kết quả này là phần ảo của x6
);

logic [31:0] rs3,rs7;
butter_fly_1 bf1_9_1 (
  .a(tmp7),
  .b(tmp6),
  .sum(rs7),  // phần ảo của x3
  .sub(rs3)   // phần ảo của x7
);


// phần thực:
assign re0 = rs0;
assign re1 = rs1;
```

```
    assign re2 = rs2;
    assign re3 = -1*tmp7_re;
    assign re4 = rs4;
    assign re5 = rs5;
    assign re6 = 32'd0;  // chỉ có phần ảo
    assign re7 = tmp7_re;

    // phần ảo:
    assign im0 = 32'd0;
    assign im1 = tmp5_im;
    assign im2 = 32'd0;
    assign im3 = rs3;
    assign im4 = 32'd0;
    assign im5 = -1*tmp5_im;
    assign im6 = rs6;
    assign im7 = rs7;

endmodule
```

2.5. KIỂM ĐỊNH BẰNG ASSERTION

2.5.1. CODE

```
`timescale 1ns/1ns
```

```
module testbench;

  // ====================== clock / reset ======================
  logic clk;
  logic rst_n;

  initial clk = 1'b0;
  always #5 clk = ~clk;

  task automatic apply_reset();
    rst_n = 1'b0;
    repeat (3) @(posedge clk);
    rst_n = 1'b1;
    @(posedge clk);
  endtask

  //
==============================================================
  // 1) UNIT TEST: full_adder
  //
==============================================================
  logic fa_a, fa_b, fa_cin;
  logic fa_sum, fa_cout;

  full_adder u_fa (
    .a(fa_a), .b(fa_b), .c_in(fa_cin),
    .sum(fa_sum), .c_out(fa_cout)
  );

  property p_full_adder;
    @(posedge clk)
      {fa_cout, fa_sum} == (fa_a + fa_b + fa_cin);
```

```
        endproperty
        assert property(p_full_adder)
          else $error("full_adder FAIL: a=%0b b=%0b cin=%0b => sum=%0b
cout=%0b",
                    fa_a,fa_b,fa_cin,fa_sum,fa_cout);

        //
==============================================================
        // 2) UNIT TEST: full_adder_32bit
        //
==============================================================
        logic [31:0] fa32_a, fa32_b;
        logic        fa32_cin;
        logic [31:0] fa32_sum;
        logic        fa32_cout;

        full_adder_32bit u_fa32 (
          .a(fa32_a), .b(fa32_b), .c_in(fa32_cin),
          .sum(fa32_sum), .c_out(fa32_cout)
        );

        property p_fa32;
          @(posedge clk)
            ({fa32_cout, fa32_sum} == ({1'b0,fa32_a} + {1'b0,fa32_b} + fa32_cin));
        endproperty
        assert property(p_fa32)
          else $error("fa32 FAIL: a=%h b=%h cin=%0b => sum=%h cout=%0b",
                    fa32_a,fa32_b,fa32_cin,fa32_sum,fa32_cout);

        //
==============================================================
        // 3) UNIT TEST: sub_32bit
```

```
        //
==============================================================
        logic [31:0] sub_a, sub_b;
        logic [31:0] sub_sum;
        logic        sub_cout;

        sub_32bit u_sub32 (
          .a(sub_a), .b(sub_b),
          .sum(sub_sum), .c_out(sub_cout)
        );

        property p_sub32;
          @(posedge clk)
            ({sub_cout, sub_sum} == ({1'b0,sub_a} + {1'b0,(~sub_b)} + 33'd1));
        endproperty
        assert property(p_sub32)
          else $error("sub32 FAIL: a=%h b=%h => sum=%h cout=%0b",
                    sub_a,sub_b,sub_sum,sub_cout);

        //
==============================================================
        // 4) UNIT TEST: butter_fly_1  (sum=a+b, sub=a-b)
        //
==============================================================
        logic [31:0] bf_a, bf_b;
        logic [31:0] bf_sum, bf_sub;

        butter_fly_1 u_bf1 (
          .a(bf_a), .b(bf_b),
          .sum(bf_sum), .sub(bf_sub)
        );
```

```
property p_bf1;
  @(posedge clk)
    (bf_sum == (bf_a + bf_b)) && (bf_sub == (bf_a - bf_b));
endproperty
assert property(p_bf1)
  else $error("bf1 FAIL: a=%h b=%h => sum=%h sub=%h",
              bf_a,bf_b,bf_sum,bf_sub);

//
==============================================================
// 5) UNIT TEST: delay  (dout = din delayed 9 cycles)
//     FIX: only assert after pipeline warmed up (>=9 cycles)
//
==============================================================
logic signed [31:0] d_din;
logic signed [31:0] d_dout;

delay u_delay (
  .clk(clk),
  .rst_n(rst_n),
  .din(d_din),
  .dout(d_dout)
);

int delay_cnt;
always_ff @(posedge clk or negedge rst_n) begin
  if (!rst_n)
    delay_cnt <= 0;
  else if (delay_cnt < 9)
    delay_cnt <= delay_cnt + 1;
end
```

```
        property p_delay9;
          @(posedge clk)
            (delay_cnt >= 9) |-> (d_dout == $past(d_din, 9));
        endproperty
        assert property(p_delay9)
          else $error("delay FAIL: din_now=%h dout=%h expected=%h",
                    d_din, d_dout, $past(d_din,9));

        //
==============================================================
        // 6) TOP TEST: fft (your module)
        //
==============================================================
        logic [31:0] x0,x1,x2,x3,x4,x5,x6,x7;
        logic [31:0] re0,re1,re2,re3,re4,re5,re6,re7;
        logic [31:0] im0,im1,im2,im3,im4,im5,im6,im7;

        fft dut (
          .clk(clk),
          .rst_n(rst_n),
          .x0(x0),.x1(x1),.x2(x2),.x3(x3),.x4(x4),.x5(x5),.x6(x6),.x7(x7),
          .re0(re0),.re1(re1),.re2(re2),.re3(re3),.re4(re4),.re5(re5),.re6(re6),.re7(re7),

.im0(im0),.im1(im1),.im2(im2),.im3(im3),.im4(im4),.im5(im5),.im6(im6),.im7(im7)
        );

        function automatic bit all_fft_outputs_zero();
          return (re0==0 && re1==0 && re2==0 && re3==0 && re4==0 && re5==0
&& re6==0 && re7==0 &&
                  im0==0 && im1==0 && im2==0 && im3==0 && im4==0 &&
im5==0 && im6==0 && im7==0);
        endfunction
```

```
task automatic check_fft_all_zero_after(input int cycles);
  int n;
  for (n = 0; n < cycles; n++) @(posedge clk);

  if (!all_fft_outputs_zero()) begin
    $error("FFT FAIL: expected all zero after %0d cycles", cycles);
    $display("re: %h %h %h %h %h %h %h %h",
             re0,re1,re2,re3,re4,re5,re6,re7);
    $display("im: %h %h %h %h %h %h %h %h",
             im0,im1,im2,im3,im4,im5,im6,im7);
    $fatal;
  end else begin
    $display("FFT PASS: all zero after %0d cycles", cycles);
  end
endtask

//
==============================================================
// Stimulus
//
==============================================================
int t;

initial begin
  // init everything (avoid X)
  rst_n   = 1'b1;

  fa_a=0; fa_b=0; fa_cin=0;
  fa32_a=0; fa32_b=0; fa32_cin=0;
  sub_a=0; sub_b=0;
  bf_a=0; bf_b=0;
```

```
d_din='0;

x0=0; x1=0; x2=0; x3=0; x4=0; x5=0; x6=0; x7=0;

apply_reset();

// -------- UNIT: full_adder exhaustive --------
$display("[TB] full_adder exhaustive");
for (t=0; t<8; t++) begin
  {fa_a,fa_b,fa_cin} = t[2:0];
  @(posedge clk);
end

// -------- UNIT: fa32 random --------
$display("[TB] full_adder_32bit random");
repeat (50) begin
  fa32_a   = $urandom();
  fa32_b   = $urandom();
  fa32_cin = $urandom_range(0,1);
  @(posedge clk);
end

// -------- UNIT: sub32 random --------
$display("[TB] sub_32bit random");
repeat (50) begin
  sub_a = $urandom();
  sub_b = $urandom();
  @(posedge clk);
end

// -------- UNIT: bf1 random --------
$display("[TB] butter_fly_1 random");
```

```
    repeat (50) begin
      bf_a = $urandom();
      bf_b = $urandom();
      @(posedge clk);
    end

    // -------- UNIT: delay ramp --------
    $display("[TB] delay ramp");
    d_din = 0;
    repeat (30) begin
      d_din <= d_din + 1;
      @(posedge clk);
    end

    // -------- TOP: FFT zero (flush đủ lâu rồi check) --------
    $display("[TB] FFT test: all zeros");
    x0=0; x1=0; x2=0; x3=0; x4=0; x5=0; x6=0; x7=0;
    check_fft_all_zero_after(40);

    // -------- TOP: FFT impulse (quan sát) --------
    $display("[TB] FFT test: impulse x0");
    x0=32'd1000; x1=0; x2=0; x3=0; x4=0; x5=0; x6=0; x7=0;
    repeat (50) @(posedge clk);

    $display("[TB] DONE.");
    $finish;
  end

endmodule
```

2.5.2. Kết quả

```
Log   Share
[2026-01-05 11:39:06 UTC] xrun -Q -unbuffered '-timescale' '1ns/1ns' '-sysv' '-access' '+rw' design.sv testbench.sv
TOOL:   xrun    23.09-s001: Started on Jan 05, 2026 at 06:39:06 EST
xrun: 23.09-s001: (c) Copyright 1995-2023 Cadence Design Systems, Inc.
    Top level design units:
        butter_fly_2
        testbench
Loading snapshot worklib.butter_fly_2:sv .................... Done
SVSEED default: 1
xcelium> source /xcelium23.09/tools/xcelium/files/xmsimrc
xcelium> run
[TB] full_adder exhaustive
[TB] full_adder_32bit random
[TB] sub_32bit random
[TB] butter_fly_1 random
[TB] delay ramp
[TB] FFT test: all zeros
FFT PASS: all zero after 40 cycles
[TB] FFT test: impulse x0
[TB] DONE.
Simulation complete via $finish(1) at time 2815 NS + 0
./testbench.sv:234     $finish;
xcelium> exit
TOOL:   xrun    23.09-s001: Exiting on Jan 05, 2026 at 06:39:08 EST  (total: 00:00:02)
Done
```

2.6. LINK VIDEO BÁO CÁO THUYẾT TRÌNH

https://drive.google.com/file/d/1qw2pst82B8Lymfs-_U9nZwYy4KXbTF3x/view?usp=drive_link